game.py

game.py185 lines

PY

README.md

README.md92 lines

MD

I am completing a VibeCoding assignment. I have attached the original game.py and README.md from my Python Pygame Lunar Lander project.
Read both files completely and implement all four tasks listed in the README:

1. Fix touchdown validation so a successful landing checks MAX_SPEED_X, MAX_SPEED_Y and MAX_ANGLE, along with the landing-pad condition.
2. Implement ship_color(fuel_ratio) so the ship's hull transitions toward red as fuel decreases.
3. Implement on_landing(score) with a visible particle or fireworks celebration after a successful landing.
4. Implement bonus_life_threshold() to return 1500 and ensure the existing bonus_awarded logic prevents duplicate bonus lives.

Preserve the existing physics, controls, terrain, both landing pads and their multipliers, fuel, levels, lives, scoring, HUD and restart behavior.
I cannot use an AI coding agent to edit my files directly, so I need to copy and paste your output into VS Code.
Give me the COMPLETE replacement contents of game.py in one Python code block. Do not give me partial snippets or ask me to edit individual functions. Base your solution on the actual attached source code; do not invent missing classes or rewrite the game from scratch.
After the code, briefly explain the changes and provide the commands to test it. Do not claim that you executed or tested the code unless you actually did.

Spawning particle bursts with varied color and speed.

python

import math

import random

import pygame

WIDTH, HEIGHT = 800, 600

STEP = 20

THRUST, ROTATE_SPEED, BURN_RATE, FUEL_MAX = 60.0, 2.4, 22.0, 300.0

MAX_SPEED_X, MAX_SPEED_Y, MAX_ANGLE = 25.0, 40.0, 0.25

```
FOOT = 12


# Fireworks state (used by on_landing, updated/drawn by Game)
PARTICLES = []
LANDING_POS = pygame.Vector2(WIDTH / 2, HEIGHT / 2)
FIREWORK_COLORS = [(255, 220, 80), (255, 120, 80), (120, 220, 255), (150, 255, 150), (255, 140, 220)]



def ship_color(fuel_ratio):
    """Return an (r, g, b) hull colour for the given fuel ratio (1.0 = full), or None for the default."""
    t = max(0.0, min(1.0, fuel_ratio))
    full, empty = (230, 230, 240), (255, 50, 40)
    return tuple(int(empty[i] + (full[i] - empty[i]) * t) for i in range(3))



def on_landing(score):
    """Called after a successful landing with the points just earned; add fireworks or bonuses here."""
    count = min(160, 40 + score // 6)  # bigger scores -> bigger show
    per_burst = max(10, count // 3)
    for dx, dy in ((-45, -50), (0, -95), (45, -50)):
        center = LANDING_POS + pygame.Vector2(dx, dy)
        color = random.choice(FIREWORK_COLORS)
        for _ in range(per_burst):
            angle = random.uniform(0, math.tau)
            speed = random.uniform(40, 170)
            life = random.uniform(0.8, 1.6)
            PARTICLES.append({
                "pos": pygame.Vector2(center),
                "vel": pygame.Vector2(math.cos(angle), math.sin(angle)) * speed,
                "life": life,
```

```
            "max_life": life,
            "color": color,
        })


def update_particles(dt):
    for p in PARTICLES:
        p["vel"].y += 120 * dt  # light gravity on sparks
        p["pos"] += p["vel"] * dt
        p["life"] -= dt
    PARTICLES[:] = [p for p in PARTICLES if p["life"] > 0]


def draw_particles(screen):
    for p in PARTICLES:
        fade = max(0.0, p["life"] / p["max_life"])
        color = tuple(int(c * fade) for c in p["color"])
        pygame.draw.circle(screen, color, (int(p["pos"].x), int(p["pos"].y)), 2 if fade < 0.5 else 3)


def bonus_life_threshold():
    """Return a score value at which the player earns an extra life, or None to disable bonus lives."""
    return 1500


def make_terrain():
    heights, y = [], random.randint(430, 520)
    for _ in range(WIDTH // STEP + 1):
        y = max(380, min(560, y + random.randint(-32, 32)))
        heights.append(y)
    pads = []
```

```
    for start, length, mult in ((random.randint(2, 12), 4, 1), (random.randint(20, 35), 3, 3)):
        for i in range(start, start + length):
            heights[i] = heights[start]
        pads.append((start * STEP, (start + length - 1) * STEP, heights[start], mult))
    return heights, pads


def ground_y(heights, x):
    x = max(0, min(WIDTH - 1, x))
    i = int(x // STEP)
    t = (x - i * STEP) / STEP
    return heights[i] * (1 - t) + heights[i + 1] * t


def wrap_angle(angle):
    return (angle + math.pi) % math.tau - math.pi


class Game:
    def __init__(self):
        self.font = pygame.font.Font(None, 26)
        self.reset()

    def reset(self):
        self.level, self.score, self.lives = 1, 0, 3
        self.bonus_awarded = 0
        self.new_round()

    def new_round(self):
        self.heights, self.pads = make_terrain()
        self.pos = pygame.Vector2(random.randint(100, 700), 70)
```

```
        self.vel = pygame.Vector2(random.uniform(-20, 20), 0)
        self.angle, self.fuel, self.thrusting = 0.0, FUEL_MAX, False
        self.state, self.message = "fly", ""
        PARTICLES.clear()

    def pad_under(self):
        for x1, x2, y, mult in self.pads:
            if x1 <= self.pos.x - 8 and self.pos.x + 8 <= x2:
                return (x1, x2, y, mult)
        return None

    def touchdown(self):
        pad = self.pad_under()
        angle = wrap_angle(self.angle)
        if (pad
                and abs(self.vel.x) <= MAX_SPEED_X
                and abs(self.vel.y) <= MAX_SPEED_Y
                and abs(angle) <= MAX_ANGLE):
            earned = int((100 + self.fuel) * pad[3])
            self.score += earned
            self.state, self.message = "landed", f"Perfect landing! +{earned}  (Space = next level)"
            LANDING_POS.update(self.pos.x, self.pos.y + FOOT)
            on_landing(earned)
            return
        self.lives -= 1
        self.state = "crashed"
        if pad is None:
            reason = "missed the pad"
        elif abs(angle) > MAX_ANGLE:
            reason = "bad angle"
        else:
```

```
            reason = "too fast"
        self.message = f"Crashed: {reason}!  " + ("Space = retry" if self.lives > 0 else "Game over - R =
restart")


    def update(self, dt, keys):
        update_particles(dt)  # keep fireworks animating even after landing
        if self.state != "fly":
            return
        threshold = bonus_life_threshold()
        if threshold and self.score // threshold > self.bonus_awarded:
            self.bonus_awarded = self.score // threshold
            self.lives += 1
        self.angle += (keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]) * ROTATE_SPEED * dt
        gravity = pygame.Vector2(0, 16 + 2 * self.level)
        self.thrusting = bool(keys[pygame.K_UP]) and self.fuel > 0
        acceleration = gravity
        if self.thrusting:
            acceleration = gravity + pygame.Vector2(math.sin(self.angle), -math.cos(self.angle)) * THRUST
            self.fuel = max(0.0, self.fuel - BURN_RATE * dt)
        self.vel += acceleration * dt
        self.pos += self.vel * dt
        self.pos.x %= WIDTH
        self.pos.y = max(-200, self.pos.y)
        if self.pos.y + FOOT >= ground_y(self.heights, self.pos.x):
            self.touchdown()


    def ship_points(self):
        cos, sin = math.cos(self.angle), math.sin(self.angle)
        local = [(0, -16), (10, 10), (-10, 10)]
        return [(self.pos.x + x * cos - y * sin, self.pos.y + x * sin + y * cos) for x, y in local]
```

```
def draw(self, screen):
    screen.fill((8, 8, 20))
    points = [(i * STEP, h) for i, h in enumerate(self.heights)]
    pygame.draw.polygon(screen, (70, 70, 85), points + [(WIDTH, HEIGHT), (0, HEIGHT)])
    pygame.draw.lines(screen, (190, 190, 200), False, points, 2)
    for x1, x2, y, mult in self.pads:
        pygame.draw.line(screen, (90, 230, 120), (x1, y), (x2, y), 5)
        label = self.font.render(f"x{mult}", True, (90, 230, 120))
        screen.blit(label, label.get_rect(midtop=((x1 + x2) / 2, y + 8)))
    if self.state != "crashed":
        if self.thrusting:
            cos, sin = math.cos(self.angle), math.sin(self.angle)
            flame = [(self.pos.x - 5 * cos - 10 * sin, self.pos.y - 5 * sin + 10 * cos),
                     (self.pos.x + 5 * cos - 10 * sin, self.pos.y + 5 * sin + 10 * cos),
                     (self.pos.x - 22 * sin, self.pos.y + 22 * cos)]
            pygame.draw.polygon(screen, (255, 170, 40), flame)
        color = ship_color(max(0.0, self.fuel) / FUEL_MAX) or (230, 230, 240)
        pygame.draw.polygon(screen, color, self.ship_points())
    else:
        pygame.draw.circle(screen, (255, 120, 40), self.pos, 24, 3)
    draw_particles(screen)
    ok_x = abs(self.vel.x) <= MAX_SPEED_X
    ok_y = abs(self.vel.y) <= MAX_SPEED_Y
    ok_a = abs(wrap_angle(self.angle)) <= MAX_ANGLE
    good, bad = (120, 240, 140), (250, 110, 100)
    lines = [
        (f"Fuel {self.fuel:5.0f}", (240, 240, 240)),
        (f"Vx {self.vel.x:6.1f}", good if ok_x else bad),
        (f"Vy {self.vel.y:6.1f}", good if ok_y else bad),
        (f"Angle {math.degrees(wrap_angle(self.angle)):5.0f}", good if ok_a else bad),
        (f"Score {self.score}  Lives {self.lives}  Level {self.level}", (240, 240, 240)),
```

```
        ]
        for i, (text, color) in enumerate(lines):
            screen.blit(self.font.render(text, True, color), (10, 8 + i * 22))
        if self.message:
            label = self.font.render(self.message, True, (255, 255, 120))
            screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT // 3)))


def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Lunar Lander")
    clock = pygame.time.Clock()
    game = Game()
    running = True
    while running:
        dt = min(clock.tick(60) / 1000, 0.05)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_r:
                    game.reset()
                elif event.key == pygame.K_SPACE and game.state == "landed":
                    game.level += 1
                    game.new_round()
                elif event.key == pygame.K_SPACE and game.state == "crashed" and game.lives > 0:
                    game.new_round()
        game.update(dt, pygame.key.get_pressed())
        game.draw(screen)
        pygame.display.flip()
```

```
    pygame.quit()



if __name__ == "__main__":
    main()
```

I haven't run this code, so please test it yourself before recording your videos.

What changed

1. **Touchdown bug:** the landing condition never checked MAX_SPEED_X. I added abs(self.vel.x) <= MAX_SPEED_X alongside the pad, MAX_SPEED_Y and MAX_ANGLE checks. A too-fast sideways landing now counts as a crash, and the message says "too fast", since the existing reason logic falls through to that.
2. **ship_color:** it blends linearly from the default hull colour (230, 230, 240) at full fuel to red (255, 50, 40) at empty fuel. The ratio is clamped to 0–1.
3. **on_landing:** it spawns three firework bursts above the touchdown point, with more particles for higher scores (so the x3 pad gives a bigger show). The function signature only receives score, so I added a module-level PARTICLES list and a LANDING_POS vector. touchdown() sets LANDING_POS just before calling on_landing. Particles are updated at the top of Game.update(), before the state != "fly" early return, so they keep animating while the ship is landed. They are drawn in draw() and cleared in new_round().
4. **bonus_life_threshold:** it returns 1500. The existing bonus_awarded bookkeeping stores score // 1500 when a life is granted, so the same threshold can't award a life twice. The check runs only while flying, so the extra life appears on the first frame of the next round after you cross the threshold.

Physics, controls, terrain, pads, fuel, levels, lives, scoring, HUD and restart are unchanged.

Commands to test

bash

pip install pygame

python game.py

Quick checks for your videos

- **Bug fix:** before your changes, drift sideways (hold Left/Right while thrusting) and land on a pad with Vx above 25 and a safe Vy and angle. It counts as a perfect landing. After the fix it should crash.
- **Ship colour:** hold Up and watch the hull turn red as fuel drops.
- **Fireworks:** land softly and level on a pad. Landing on the x3 pad should show the bigger burst.
- **Bonus life:** to test it quickly, temporarily change return 1500 to a small number like return 200, check that lives go up by 1 once per threshold crossing, then change it back to 1500.

game.py

game.py228 lines

PY

Please perform a final, thorough review of my updated Lunar Lander assignment. I have attached my current game.py and README.md.
Check the entire code against every requirement in README.md, especially these four tasks:

1. Touchdown validation: A successful landing must satisfy the landing-pad condition and all three limits: MAX_SPEED_X, MAX_SPEED_Y and MAX_ANGLE.
2. Ship color: ship_color(fuel_ratio) must smoothly transition the hull color toward red as fuel decreases and handle out-of-range fuel ratios safely.
3. Landing celebration: on_landing(score) must produce a visible particle/fireworks effect only after a successful landing. Check that particles are initialized, updated, drawn and cleaned up correctly.
4. Bonus lives: bonus_life_threshold() must return 1500. Verify that the existing bonus_awarded logic awards extra lives at the correct score thresholds without repeatedly awarding the same bonus.

Also check the entire program for syntax errors, potential runtime errors, incorrect function calls, and regressions in existing functionality, including physics, controls, fuel, terrain, both landing pads and their multipliers, scoring, lives, levels, HUD, crashes, resets and restart behavior.
IMPORTANT OUTPUT INSTRUCTIONS:

- If you find ANY bugs, errors, missing requirements or necessary corrections, fix them and provide the COMPLETE corrected contents of game.py in ONE Python code block.
- Do not give me partial snippets, patches, or instructions to manually edit individual functions. I need to copy and paste the entire corrected file into VS Code.
- Base the solution on my attached source files. Preserve the existing game structure and functionality. Do not rewrite the game from scratch or add unrelated features.
- If the code is already correct, do not unnecessarily rewrite it. Clearly state that no code changes are needed and explain which checks still require me to test the game manually.
- Do not claim that you executed the code or verified gameplay unless you actually did.
- If you provide corrected code, ensure bonus_life_threshold() returns 1500 in the final version.

My goal is to finalize this assignment, so prioritize correctness and compatibility over adding new features.

Verifying boundary cases in physics and landing-pad placement.

No code changes are needed. I read through your current game.py against the README and found no syntax errors, runtime errors, wrong calls or regressions. I haven't run it, so everything below comes from reading the code, not from playing it.

The four tasks

1. **Touchdown validation:** The success condition checks the pad, abs(self.vel.x) <= MAX_SPEED_X, abs(self.vel.y) <= MAX_SPEED_Y and abs(angle) <= MAX_ANGLE, using the wrapped angle. If any of these fails, it falls through to the crash branch. A sideways slam on a pad now gives “Crashed: too fast!”, because the reason logic falls through to that message when the pad and angle are fine.
2. **ship_color:** The ratio is clamped to 0–1, so values like -0.1 or 1.2 can’t produce invalid RGB values. It blends linearly from (230, 230, 240) at full fuel to (255, 50, 40) at empty fuel. It returns a non-empty tuple, so the or (230, 230, 240) fallback in draw never triggers wrongly.
3. **Fireworks:**
    - **Spawn:** on_landing is called only from the success branch of touchdown(), so crashes never spawn particles. LANDING_POS is set just before the call.
    - **Update, draw and cleanup:** update_particles runs at the top of Game.update(), before the state != "fly" early return, so sparks keep animating while landed. draw_particles runs after the ship and before the HUD, and expired particles are removed each frame. new_round() clears everything, and reset() goes through new_round(), so R, Space-to-continue and retry all start clean.
    - **Types:** Circle centers are cast to int, and colors are int tuples scaled by a fade factor in 0–1.
4. **Bonus lives:** bonus_life_threshold() returns 1500. update() awards a life when score // 1500 > bonus_awarded and then sets bonus_awarded to that value, so the same threshold can’t pay out twice. The maximum score from one landing is (100 + 300) * 3 = 1200, which is below 1500, so one landing can never cross two thresholds and the single +1 life is always correct. reset() zeroes bonus_awarded along with the score.

Other checks: Physics, controls, fuel (clamped at 0, thrust cuts out when empty), terrain, both pads (x1 and x3) and their scoring, lives, levels, HUD and restart behavior match the original. The only new module-level state is PARTICLES, LANDING_POS and FIREWORK_COLORS, and pygame.Vector2 doesn’t need pygame.init(), so creating them at import is safe.

Two things to know

- **Bonus life timing:** The life is added on the first flying frame of the next round after you cross 1500, not at the moment of landing. This follows the bookkeeping that was already in update(), so it’s expected. Don’t be surprised that the HUD’s Lives count doesn’t change on the landing screen.
- **Top-of-screen clamp:** The README says the ship “never moves past the top of the screen,” but the provided starter code clamps at self.pos.y = max(-200, self.pos.y), which lets it go 200 px above the screen. That line was in your original file and isn’t one of the four tasks, so I left it alone. If your instructor expects a hard top limit, ask them before changing it, because it affects the physics.

What you still need to check by running it

- Run python game.py with a recent pygame 2.x (pip install --upgrade pygame), since I haven’t confirmed your installed version.
- Land on a pad with Vx above 25 and a safe Vy and angle, and confirm it crashes.

- Land softly on both pads and confirm the fireworks appear, and that nothing appears after a crash.
- Watch the hull turn red as fuel drains.
- For the bonus life, temporarily change the threshold to something small like 200 to confirm one life per crossing, then set it back to 1500 before submitting.
- Press R and Space repeatedly to confirm no particles linger.

Claude is AI and can make mistakes. Please double-check responses.